Lab 1

CEG 3156 - High Level Computer Design

Winter 2026
School of Computer Engineering and Computer Science
University of Ottawa

Student Name and number: Mann Patel 300363879
Student Name and number: Surya Vasudev 300358602
Submission Date: February 11th 2026
Group: 4

Table of Contents

Introduction ..... -3-
Explaining each file ..... -4-
VHDL Simulations & Diagrams ..... -6-
    Floating-point multiplier ..... -6-
    Floating Point Adder ..... -7-
Conclusion and Problems ..... -9-
VHDL Code - GitHub Repository ..... -10-

Introduction

The objective of this laboratory is to design and implement a floating-point adder and multiplier using VHDL structural modeling. The design utilizes a custom 16-bit format consisting of a single sign bit, a 7-bit exponent, and an 8-bit mantissa to perform arithmetic operations.

Floating-point addition requires comparing exponents and right-shifting the smaller number's mantissa to align the binary points before adding the values. In contrast, multiplication is performed by adding the exponents (subtracting the bias) and directly multiplying the mantissas while calculating the new sign bit. Both operations conclude with a normalization step, where the result is shifted and the exponent adjusted to ensure the number fits the standard format.

Explaining each file

File Name	Description
bidirectional_shifter.vhdl	Performs a bidirectional shift on the input mantissa; used for both addition (alignment) and multiplication (normalization).
bigALU.vhdl	Implements an 18-bit ALU with addition, subtraction, and comparison operations.
controlUnit_Mult.vhdl	Control unit for the multiplier. A simple FSM that sequences the multiplication steps and controls the datapath.
controlUnit.vhdl	Control unit for the adder. A simple FSM that sequences the steps of addition and controls the datapath.
counter.vhdl	Implements a simple counter used for tracking normalization shifts and rounding bits.
fpAdd.vhdl	Top-level floating-point adder entity. Connects the control unit and datapath.
fpMult.vhdl	Top-level floating-point multiplier entity. Connects the control unit and datapath.
mux_2x1_16bit.vhdl	2x1 multiplexer for 16-bit inputs. Used for mantissa selection.
mux_2x1.vhdl	Generic 2x1 multiplexer.
nbit_comparator.vhdl	Comparator for n-bit inputs, used to determine the larger exponent.
nbit_register.vhdl	Generic n-bit register for storing mantissas and exponents.
onebitcomparator.vhdl	1-bit comparator used for sign bit logic.
quad_n_mux_selector.vhdl	Selector for 4 n-bit inputs, used in the control unit.
ripple_adder.vhdl	Ripple adder for n-bit inputs, used for mantissa addition and exponent math.
rshift_byte.vhdl	Right shift logic for a byte, used for shifting product results.

smallALU.vhdl	1-bit ALU with AND, OR, XOR, and NOT operations.
srlatch.vhdl	Shift register with latch, used for storing and shifting mantissas during normalization.

VHDL Simulations & Diagrams

Floating-point multiplier

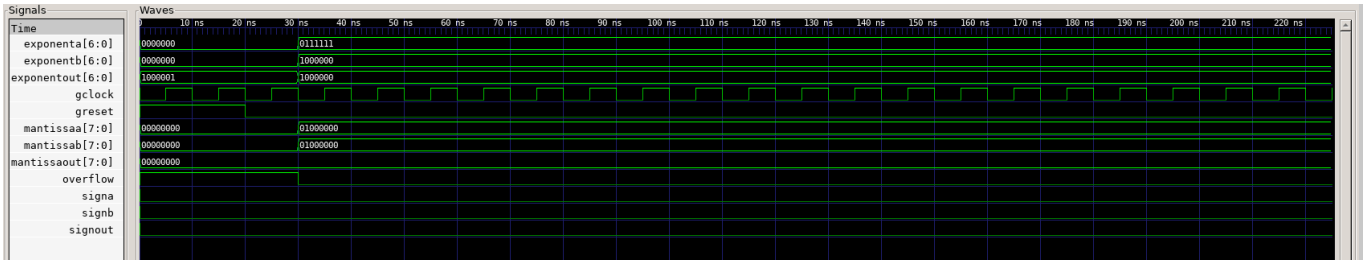

Figure 1: Timing simulation of the floating-point multiplier. Verifies exponent summation and mantissa product. Note: artifacts may be present in LSBs due to normalization logic.

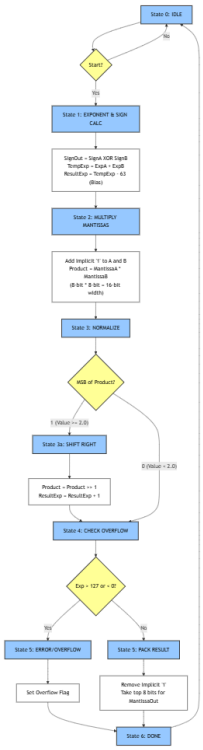

Figure 2: FSM chart for the floating-point multiplier (XOR Sign, Subtract Bias, Multiply Loop).

Floating Point Adder

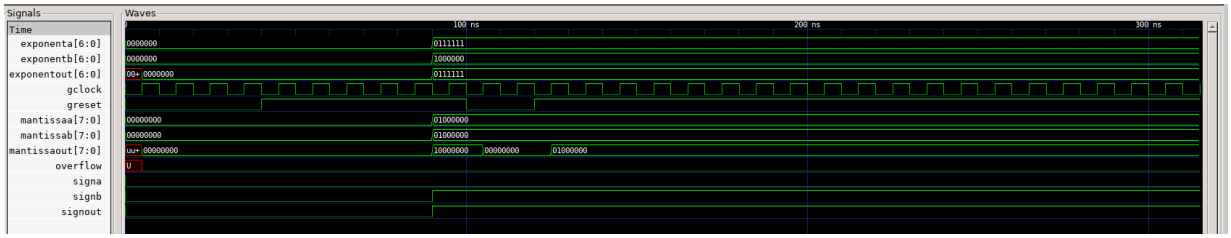

Figure 3: This diagram outlines the control logic for managing the steps of floating-point addition.

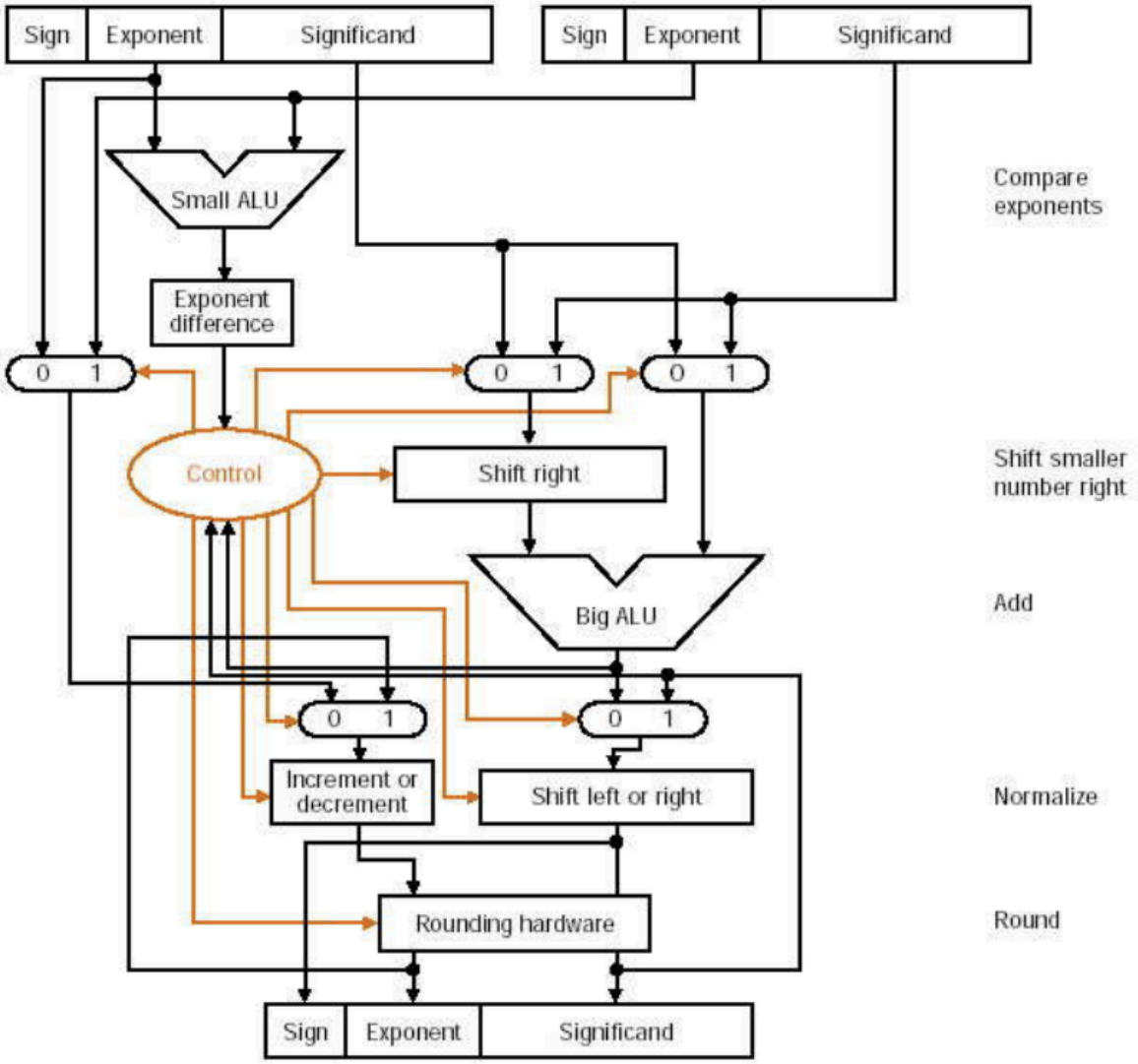

Figure 4: This diagram outlines the simulation of floating-point addition.

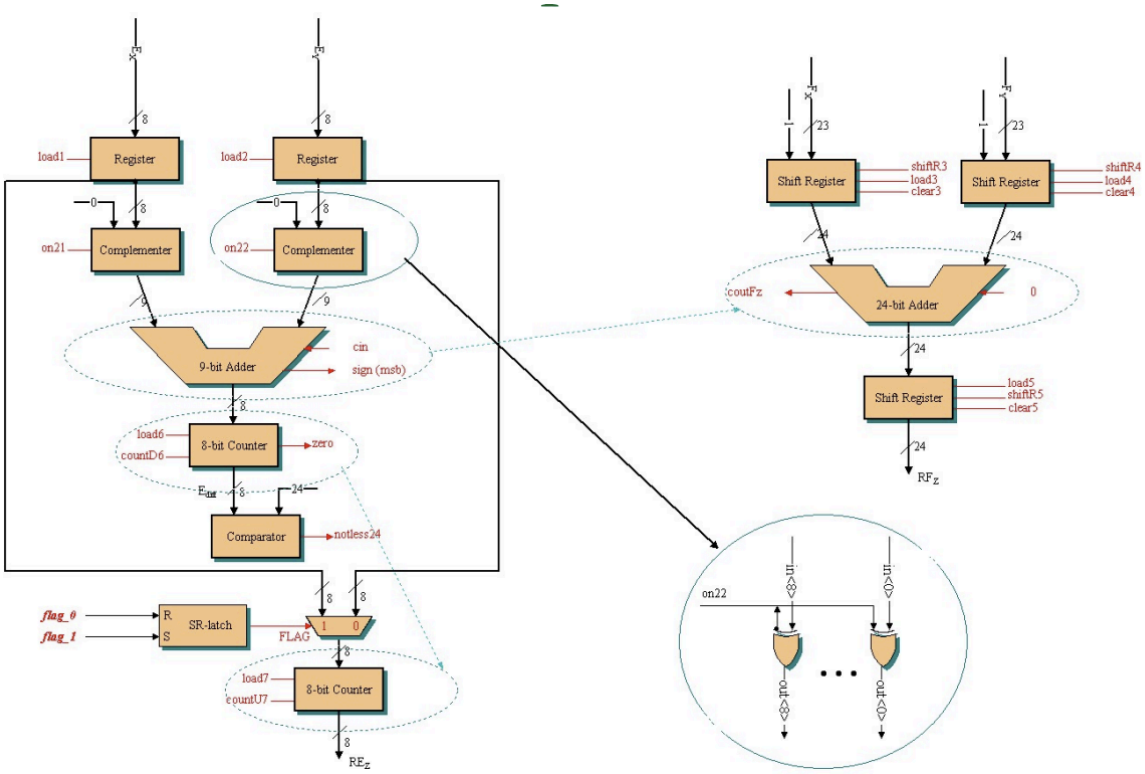

Figure 5: This diagram outlines the datapath for the floating-point adder.

Conclusion and Problems

The most significant challenge faced during this laboratory was the implementation of the floating-point multiplier unit. While the floating-point adder functions as expected, the multiplier currently fails to produce the correct results during timing simulations. The issue likely stems from the normalization logic applied after the mantissa multiplication; specifically, shifting the 16-bit product result back into the 8-bit range appears to be corrupting the data.

Tracing errors through the schematic design was time-consuming, and due to these debugging delays, we were unable to resolve the multiplier errors before the submission deadline.

In conclusion, the objective of this laboratory was to design and realize a floating-point arithmetic unit using VHDL. We successfully implemented and verified the floating-point adder, demonstrating an understanding of exponent comparison and mantissa alignment. However, we were unable to demonstrate a fully functional floating-point multiplier due to persistent logic errors in the normalization stage. Despite the issues with the multiplication unit, the lab provided valuable insight into the complexities of floating-point arithmetic and the challenges of hardware design and debugging.

VHDL Code - GitHub Repository

CLICK FOR GitHub Repository